

Course 4: Multiprotocol Development

USP Training Series

USP Development Team

Universal Software Platform

November 21, 2025

Course Overview

- 1 Introduction
- 2 Ping-Pong Communication
- 3 PER Testing
- 4 LoRa Ranging
- 5 Multiprotocol Applications
- 6 Hands-On Labs
- 7 Summary

Course 4: Multiprotocol Development

Learning Objectives

- Implement ping-pong bidirectional communication
- Measure and analyze Packet Error Rate (PER)
- Use LoRa ranging for distance measurement
- Develop multiprotocol applications with RAC
- Integrate custom protocols with LoRaWAN

Hands-On Labs

- Lab 4.1: Ping-Pong Communication
- Lab 4.2: PER Testing
- Lab 4.3: LoRa Ranging
- Lab 4.4: Multiprotocol Application

Prerequisites

- Completion of Course 3 (USP & RAC Architecture)
- Understanding of RAC transaction model
- Familiarity with LR1120/LR1121 radio operations
- Knowledge of LoRa modulation parameters
- Two development boards for bidirectional testing

Hardware Requirements

Two Xiao nRF54L15 + LR1120 boards required for Labs 4.1-4.3

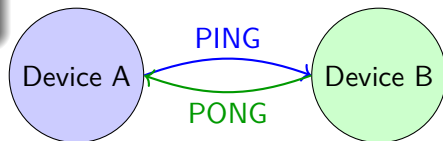
Ping-Pong Communication Overview

What is Ping-Pong?

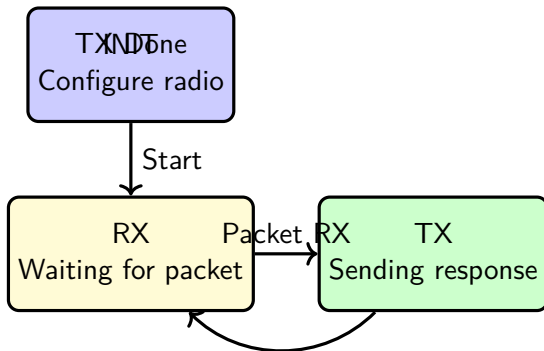
Bidirectional LoRa communication where two devices alternately transmit and receive messages.

Use Cases:

- Range testing
- Link quality assessment
- Protocol development
- Bidirectional data exchange



Ping-Pong State Machine



Key Parameters:

- Timeout for RX mode (typically 5-10 seconds)
- TX/RX turnaround time
- Payload format (sequence number, RSSI, SNR)

Ping-Pong Implementation

```
typedef enum {
    STATE_RX,
    STATE_TX,
} pingpong_state_t;

static pingpong_state_t state = STATE_RX;
static uint32_t ping_count = 0;

void pingpong_event_handler(void) {
    if (radio_event == RADIO_RX_DONE) {
        // Packet received - extract payload
        uint8_t payload[256];
        uint8_t size;
        lr11xx_radio_get_rx_buffer_status(&size, &
            offset);
        lr11xx_regmem_read_buffer8(payload, offset,
            size);

        ping_count = (payload[0] << 8) | payload[1];
```

Ping-Pong TX Implementation

```
void pingpong_send_response(void) {
    uint8_t payload[4];

    // Increment ping count
    ping_count++;

    // Build payload
    payload[0] = (ping_count >> 8) & 0xFF;
    payload[1] = ping_count & 0xFF;
    payload[2] = last_rssi;
    payload[3] = last_snr;

    // Send via RAC
    rac_params_t params = {
        .priority = 128,
        .pre_callback = pingpong_tx_pre,
        .post_callback = pingpong_tx_post,
        .context = payload,
    };
};
```


Packet Error Rate (PER) Testing

What is PER?

PER measures the percentage of packets that fail to be received correctly.

$$\text{PER} = \frac{\text{Packets Lost}}{\text{Packets Sent}} \times 100\%$$

Why Measure PER?

- Evaluate link quality under real conditions
- Determine maximum communication range
- Optimize modulation parameters (SF, BW, CR)
- Validate antenna performance
- Compare different environments (urban, rural, indoor)

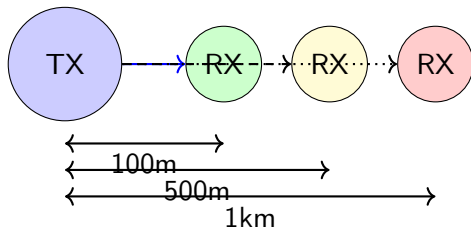
PER Test Setup

Test Configuration:

- Fixed transmitter (TX)
- Mobile receiver (RX)
- Continuous transmission
- Sequence number in payload
- RSSI/SNR logging

Typical Test:

- Send 1000 packets
- Increase distance
- Record lost packets
- Calculate PER



Distance	PER	Link
100m	0.1%	Excellent
500m	2.5%	Good
1km	15%	Poor

PER Transmitter Implementation

```
static uint32_t seq_number = 0;
static uint32_t total_packets = 0;

void per_tx_thread(void) {
    while (1) {
        // Build packet with sequence number
        uint8_t payload[8];
        payload[0] = (seq_number >> 24) & 0xFF;
        payload[1] = (seq_number >> 16) & 0xFF;
        payload[2] = (seq_number >> 8) & 0xFF;
        payload[3] = seq_number & 0xFF;
        payload[4] = 0xAA; // Magic marker
        payload[5] = 0x55;

        // Transmit via RAC
        per_tx_packet(payload, sizeof(payload));

        seq_number++;
        total_packets++;
    }
}
```

PER Receiver Implementation

```
static uint32_t last_seq = 0;
static uint32_t total_rx = 0;
static uint32_t total_lost = 0;

void per_rx_event_handler(void) {
    if (radio_event == RADIO_RX_DONE) {
        uint32_t seq = (payload[0] << 24) |
                        (payload[1] << 16) |
                        (payload[2] << 8) | payload[3];

        // Check for lost packets
        if (seq > last_seq + 1) {
            uint32_t lost = seq - last_seq - 1;
            total_lost += lost;
            printk("Lost %u packets!\n", lost);
        }

        last_seq = seq;
        total_rx++;
    }
}
```

PER Analysis and Interpretation

Link Budget Correlation

PER results validate link budget calculations:

- **PER < 1%:** Excellent link, margin for fading
- **PER 1-5%:** Good link, acceptable for most applications
- **PER 5-10%:** Marginal link, retransmissions needed
- **PER > 10%:** Poor link, move closer or increase SF

Optimization Strategies:

- 1 Increase Spreading Factor (SF) → lower data rate, higher sensitivity
- 2 Use lower bandwidth → better SNR
- 3 Increase TX power → higher link budget
- 4 Improve antenna positioning
- 5 Enable Forward Error Correction (higher CR)

LoRa Ranging Technology

What is Ranging?

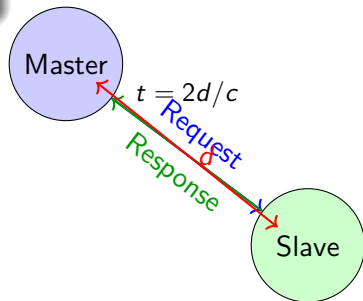
LoRa ranging uses Time-of-Flight (ToF) measurements to determine the distance between two devices.

How it Works:

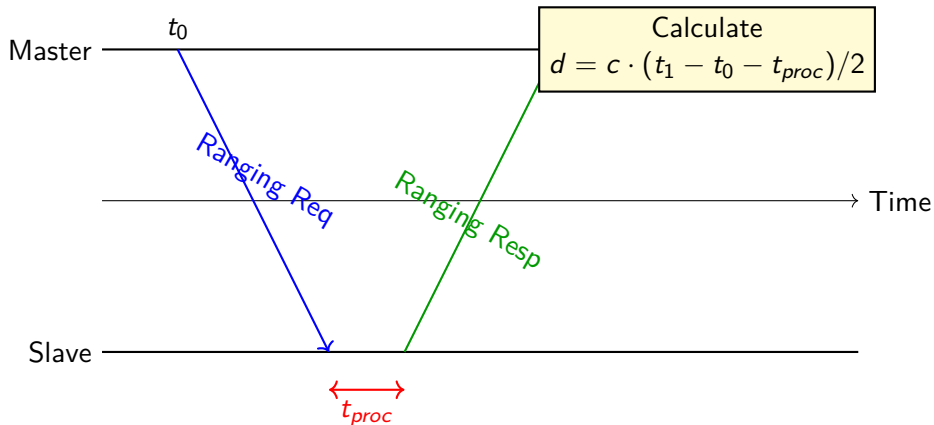
- 1 Master sends ranging request
- 2 Slave responds with ranging packet
- 3 Radio measures round-trip time
- 4 Distance calculated: $d = \frac{c \cdot t}{2}$

Accuracy:

- Typical: $\pm 10\text{-}30\text{m}$
- Best case: $\pm 5\text{m}$ (open field)
- Worst case: $\pm 50\text{m}$ (urban multipath)



Ranging Packet Exchange



Key Timing:

- t_0 : Master sends request
- t_{proc} : Slave processing time (known, calibrated)

Ranging Implementation - Master

```
void ranging_master_start(void) {  
    // Configure ranging parameters  
    lr11xx_radio_lora_ranging_params_t params = {  
        .address = SLAVE_ADDRESS,  
        .request_count = 10,    // Average 10  
                                measurements  
        .role = LR11XX_RADIO_LORA_RANGING_MASTER,  
    };  
  
    // Set ranging mode  
    lr11xx_radio_set_lora_ranging(&params);  
  
    // Start ranging transaction via RAC  
    rac_params_t rac_params = {  
        .priority = 200,  
        .pre_callback = ranging_pre_callback,  
        .post_callback = ranging_post_callback,  
    };  
}
```


Ranging Implementation - Slave

```
void ranging_slave_init(void) {  
    // Configure as slave  
    lr11xx_radio_lora_ranging_params_t params = {  
        .address = MY_ADDRESS,  
        .role = LR11XX_RADIO_LORA_RANGING_SLAVE,  
    };  
  
    lr11xx_radio_set_lora_ranging(&params);  
  
    // Start continuous RX for ranging requests  
    lr11xx_radio_set_rx(RX_CONTINUOUS);  
}  
  
void ranging_slave_event_handler(void) {  
    if (radio_event == RADIO_RANGING_REQUEST_VALID) {  
        // Radio automatically responds  
        printf("Ranging request received\n");  
    }  
}
```

Extract Ranging Results

```
void ranging_result_handler(void) {
    lr11xx_radio_lora_ranging_result_t result;

    lr11xx_radio_get_lora_ranging_result(&result);

    if (result.status ==
        LR11XX_RADIO_RANGING_STATUS_VALID) {

        float distance_m = result.distance_raw * 150.0
            f
                               / 4096.0f;  // Convert to
                                       meters

        int16_t rssi = result.rssi;

        printf("Distance: %.2f m, RSSI: %d dBm\n",
            distance_m, rssi);

        // Store for averaging
    }
}
```

Ranging Applications

Asset Tracking:

- Locate devices without GPS
- Indoor positioning
- Warehouse inventory
- Livestock tracking

Proximity Detection:

- Social distancing
- Collision avoidance
- Access control
- Geofencing

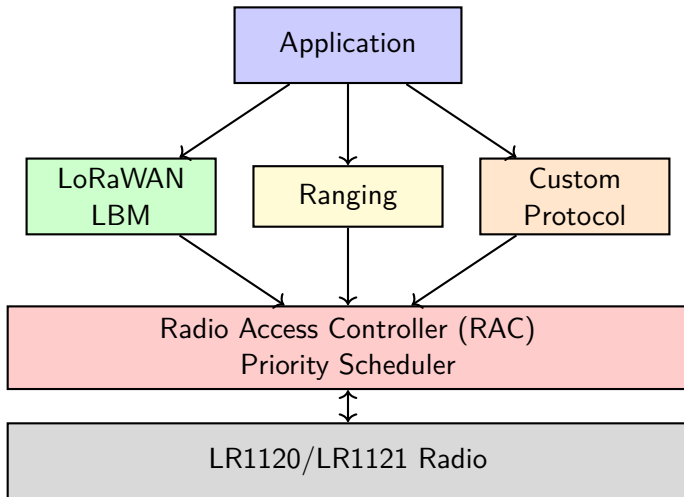
Advantages over GPS:

- Works indoors
- Lower power consumption
- No satellite dependency
- Lower cost

Limitations:

- Requires line-of-sight
- Affected by multipath
- Limited to LR11xx radios
- 2-device measurement only

Multiprotocol Architecture with RAC



Multiprotocol Use Cases

LoRaWAN + Ranging

Asset tracker that reports location via LoRaWAN and uses ranging for proximity detection.

LoRaWAN + Custom Protocol

Smart meter with LoRaWAN uplinks and local mesh networking for meter-to-meter communication.

LoRaWAN + PER Testing

Deployed sensor that performs periodic link quality tests while maintaining normal operation.

Key Design Principles:

- Assign appropriate priorities (LoRaWAN highest)
- Respect duty cycle limits across all protocols
- Handle conflicts gracefully (RAC queuing)

Multiprotocol Priority Assignment

```
// Priority levels (0 = highest, 255 = lowest)
#define PRIORITY_LORAWAN_EMERGENCY    0
#define PRIORITY_LORAWAN_CONFIRMED    50
#define PRIORITY_LORAWAN_UNCONFIRMED  100
#define PRIORITY_RANGING                150
#define PRIORITY_CUSTOM_PROTOCOL       200
#define PRIORITY_HEARTBEAT             250

void configure_protocols(void) {
    // LoRaWAN always has highest priority
    lbm_set_priority(PRIORITY_LORAWAN_CONFIRMED);

    // Ranging for asset tracking
    ranging_set_priority(PRIORITY_RANGING);

    // Custom protocol for local mesh
    custom_proto_set_priority(
        PRIORITY_CUSTOM_PROTOCOL);
}
```

Handle Protocol Conflicts

```
void ranging_request(void) {
    rac_params_t params = {
        .priority = PRIORITY_RANGING,
        .pre_callback = ranging_pre,
        .post_callback = ranging_post,
    };

    rac_status_t status = rac_add(&params, &handle);

    if (status == RAC_STATUS_BUSY) {
        // Higher priority TX in progress
        printk("Ranging deferred\n");

        // Retry later or queue request
        k_work_schedule(&ranging_retry_work,
                        K_SECONDS(5));
    } else if (status == RAC_STATUS_OK) {
        printk("Ranging started\n");
    }
}
```

Duty Cycle Management

Challenge

Multiple protocols must share limited duty cycle budget (typically 1% in EU868).

Solution Strategies:

- 1 **Global Budget:** Track total airtime across all protocols
- 2 **Per-Protocol Limits:** Allocate budget to each protocol
- 3 **Dynamic Allocation:** Unused budget can be borrowed
- 4 **Adaptive Scheduling:** Reduce custom protocol activity if LoRaWAN needs bandwidth

Example Budget Allocation

- LoRaWAN: 0.7% (70% of 1% budget)
- Ranging: 0.2% (20%)
- Custom: 0.1% (10%)

Lab 4.1: Ping-Pong Communication

Objective

Implement bidirectional LoRa communication between two devices with automatic turnaround.

Tasks:

- 1 Configure one board as initiator, one as responder
- 2 Implement RX $\rightarrow$ TX $\rightarrow$ RX cycle
- 3 Include sequence number, RSSI, SNR in payload
- 4 Add timeout handling for lost packets
- 5 Test at various distances (10m, 50m, 100m)

Deliverables:

- Working ping-pong firmware for both boards
- Serial output showing packet exchange
- Analysis of round-trip time

Lab 4.2: PER Testing

Objective

Measure Packet Error Rate over distance and analyze link quality.

Tasks:

- 1 Implement TX firmware with sequence numbers
- 2 Implement RX firmware with lost packet detection
- 3 Test at SF7, SF10, SF12 (compare sensitivity)
- 4 Record PER at 100m, 250m, 500m, 1km
- 5 Generate PER vs. Distance graph

Deliverables:

- PER test firmware (TX and RX)
- Test results table
- Graph showing PER vs. distance for different SF

Lab 4.3: LoRa Ranging

Objective

Implement ranging to measure distance between two devices.

Tasks:

- 1 Configure master device to initiate ranging
- 2 Configure slave device to respond to requests
- 3 Parse ranging results and calculate distance
- 4 Average multiple measurements for accuracy
- 5 Compare ranging distance to actual measured distance

Deliverables:

- Ranging master and slave firmware
- Distance measurements at known separations
- Error analysis (ranging vs. actual)

Lab 4.4: Multiprotocol Application

Objective

Develop application that combines LoRaWAN uplinks with ranging-based geofencing.

Scenario: Asset tracker that sends location via LoRaWAN and triggers alert if ranging detects proximity to another tagged device (collision warning).

Tasks:

- 1 Implement LoRaWAN uplinks every 60 seconds
- 2 Implement ranging every 10 seconds
- 3 Use RAC to schedule both protocols
- 4 Trigger immediate LoRaWAN uplink if ranging $< 50\text{m}$
- 5 Handle priority conflicts (LoRaWAN takes precedence)

Deliverables:

- Multiprotocol application firmware

• Demonstration of proximity alert

Course 4 Summary

Key Takeaways

- Ping-pong enables bidirectional LoRa communication
- PER testing quantifies link quality over distance
- LoRa ranging provides GPS-free distance measurement
- RAC enables seamless multiprotocol operation
- Priority assignment critical for protocol coexistence
- Duty cycle must be managed across all protocols

Next Course:

- Course 5: Hardware Integration
- Device tree configuration
- TX power calibration
- HAL porting for custom boards

Additional Resources

Documentation:

- LR1120/LR1121 Datasheet (Semtech)
- AN1200.48: LoRa Ranging
- LoRa Alliance Technical Recommendations
- USP RAC Architecture Guide

Example Code:

- `samples/lora/ping_pong/`
- `samples/lora/per_test/`
- `samples/lora/ranging/`
- `samples/multiprotocol/lorawan_ranging/`

Lab Solutions:

- `doc/training/Lab_Solutions.md`

Thank You!

Questions?

Next: Course 5 - Hardware Integration

Contact: `usp-support@example.com`